FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

FECAP

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Ana Julia Bernardo Quintans - 26028857

Clayton Santos Ferreira – 26029419

Leonardo Zachello Encinas - 26028745

Riane Toscano - 26029235

Ricardo Branco Rios - 26029360

PROJETO INTEGRADO
MODELAGEM DE SOFTWARE E ARQUITETURA DE SISTEMAS

PROFESSOR(A) ORIENTADOR(A): AIMAR MARTINS LOPES

São Paulo

2026

Sumário

1 INTRODUÇÃO **3**

1.1 – Processo de desenvolvimento de Software 3

1.2 Apresentação 3

2. DESIGN SPRINT – Ideação e prototipação do desafio **3**

2.1 Desafio 3

2.2 Entender Mapear 4

2.2.1 – Entendimento e observação. COMO PODEMOS? 4

2.2.2 – Pesquisa Desk 4

2.3 Ideação – desenho da solução (trilha do usuário) 4

2.4 Prototipagem 4

3.REQUISITOS DE SISTEMA **4**

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE 4

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE 5

4. CASOS DE USO **6**

5. DIAGRAMA DE CLASSE **6**

6. ARQUITETURA DO SISTEMA **6**

6.1. Visão Geral do Sistema 6

6.2. Arquitetura Geral 6

6.3. Componentes Arquiteturais e Camadas 6

6.4. Diagrama da Arquitetura 7

6.5. Organização de Pastas e Módulos 8

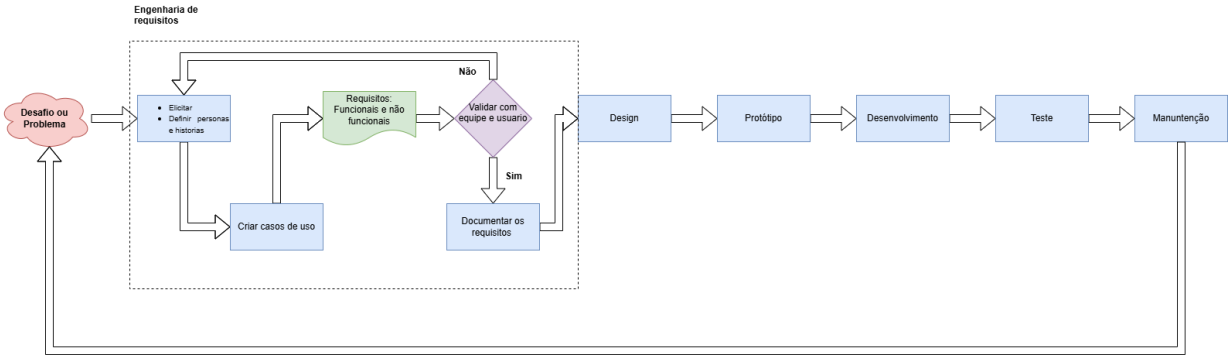
6.6. Integrações Externas (se houver) 8

6.7. Ambiente e Implantação 9

8. REFERÊNCIAS BIBLIOGRÁFICAS **9**

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produtos e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote. Tendo.....

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

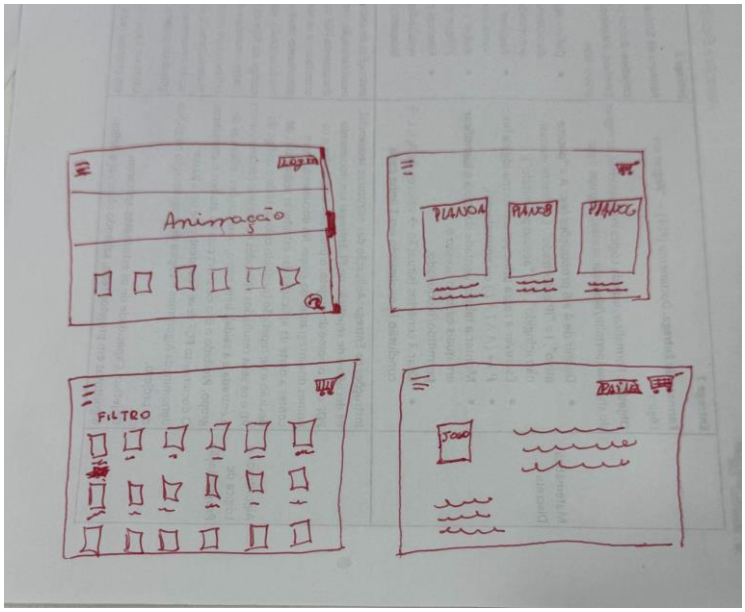
Reuniões frequentes para alinhamento de ideias, planejamento e implementação das ideias.

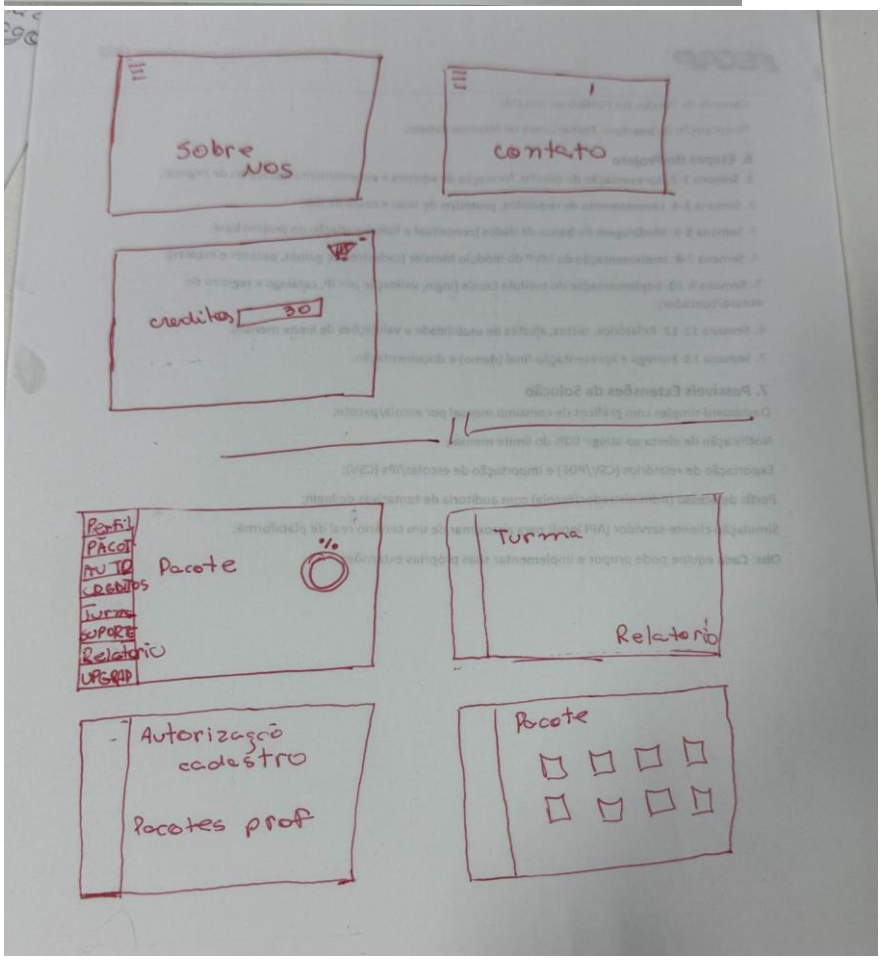
Como podemos?

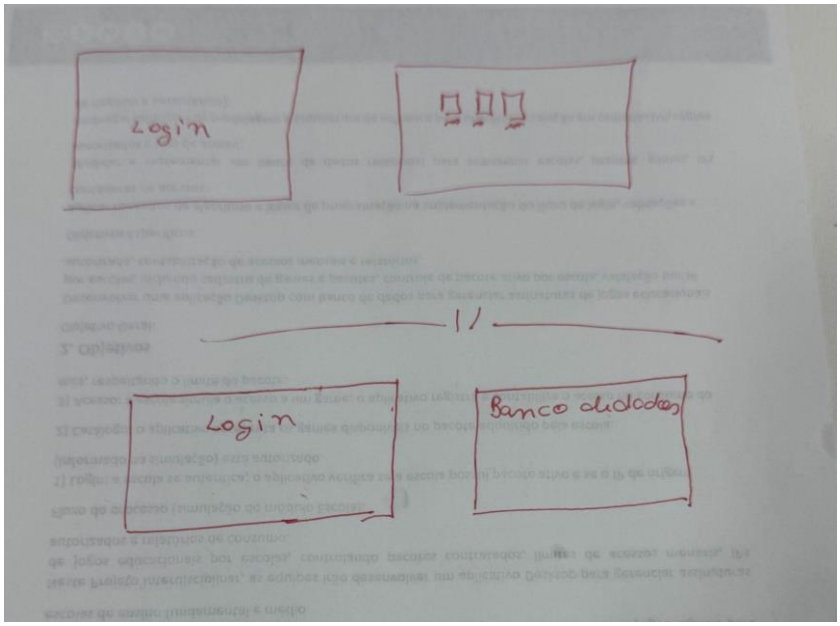
- Fazer um site atrativo
- Fazer um site auto explicativo
- Fazer Banco de dados
- Como autorizar por IP

2.2.2 – Pesquisa Desk

2.3 Ideação – desenho da solução (trilha do usuário)







2.4 Prototipagem

3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Cadastro de Games;
Descrição	O sistema deve permitir o cadastro de games (CRUD) com nome, descrição, tema, faixa etária e status;
Entradas	id_game, nome, descricao, tema, faixa_etaria, status;
Fonte	Administrador por meio da interface do sistema;
Saídas	Confirmação de operação, listagem de games, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir games, realizando validações e persistindo os dados em banco.

RFS02	
Função	Cadastro de Pacotes;
Descrição	O sistema deve permitir o cadastro de Pacotes (CRUD) com lista de games, limite de acessos mensais e preço;
Entradas	id_pacote, nome, limite_Acessos_mensal, preco_mensal;
Fonte	Administrador por meio da interface do sistema;

Saídas	Confirmação de operação, listagem de pacotes, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir pacotes, realizando validações e persistindo os dados em banco.

RFS03	
Função	Cadastro de Escolas;
Descrição	O sistema deve permitir o cadastro de escolas (CRUD) com dados básicos, pacote ativo e IPs autorizados;
Entradas	id_escola, nome, login, senha, pacote_ativo, ips_autorizados;
Fonte	Administrador por meio da interface do sistema;
Saídas	Confirmação de operação, listagem de escolas, exibição de dados e mensagens de erro;
Ação	O sistema deve permitir cadastrar, consultar, atualizar e excluir escolas, realizando validações e persistindo os dados em banco.

RFS04	
Função	Login e Validação de Usuário;
Descrição	O sistema deve realizar login da escola, validando pacote ativo e IP de origem;
Entradas	login, senha;
Fonte	Escola por meio da interface do sistema;
Saídas	Acesso concedido à plataforma, acesso negado (com motivo), mensagens de erro/validação, registro de acesso;
Ação	O sistema deve autenticar o usuário, validar credenciais, verificar a existência de pacote ativo dentro da validade e conferir se o acesso está sendo feito a partir de um IP autorizado. Se todas as condições forem atendidas, o acesso é permitido.

RFS05	
Função	Catálogo de Games;
Descrição	O sistema deve exibir o catálogo de games disponíveis conforme o pacote da escola;
Entradas	id_escola, filtros opcionais;

Fonte	Sistema (estado de autenticação) e banco de dados de games;
Saídas	Listagem de games disponíveis conforme seu pacote ativo e filtros (se escolhidos);
Ação	O sistema deve permitir que usuários autenticados tenham acesso ao catálogo de games realizando consulta ao banco de dados e exibindo apenas jogos ativos no pacote do usuário. Deve permitir a utilização de filtros para facilitar a busca.

RFS06	
Função	Registro de Acesso;
Descrição	O sistema deve registrar acessos aos games, contabilizando consumo mensal e bloqueando quando atingir o limite;
Entradas	id_acesso, id_escola, id_game, data e hora, id_ip, limite_acessos_mes;
Fonte	Próprio sistema ao identificar acesso à um game.
Saídas	Registro de acesso, atualização da contagem mensal e liberação ou bloqueio de acesso;
Ação	O sistema deve registrar o acesso ao game e atualizar a contagem mensal para seu respectivo usuário e bloquear o acesso se não tiver mais acessos disponíveis.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RNFS01	
Função	Modo local;
Descrição	A aplicação deve funcionar offline, para fins de protótipo;

Entradas	Configuração do Sistema, Caminho do Banco de Dados Local;
Fonte	Sistema, Máquina do Usuário;
Saídas	Sistema funcionando sem internet, Dados acessíveis localmente;
Ação	O sistema deve operar de forma offline, utilizando banco de dados local, sem dependência de conexão externa.

RNFS02	
Função	Interface intuitiva;
Descrição	Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;
Entradas	Interações do Usuário (Cliques, navegação);
Fonte	Usuário;
Saídas	Navegação simples e compreensível, redução de erros de uso;
Ação	O sistema deve apresentar uma interface clara, organizada e de fácil utilização, permitindo que usuários realizem operações sem necessidade de treinamento avançado.

RNFS03	
Função	Sistema veloz;
Descrição	Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;
Entradas	Requisições do usuário, Consultas ao banco;
Fonte	Usuário, Sistema;
Saídas	Respostas rápidas, Baixo tempo de carregamento;
Ação	O sistema deve responder às ações do usuário em tempo reduzido, garantindo desempenho adequado mesmo com múltiplos registros.

RNFS04	
Função	Código organizado;
Descrição	Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);
Entradas	Código-fonte do sistema;
Fonte	Desenvolvedor;
Saídas	Código legível e modular;
Ação	O sistema deve ser desenvolvido seguindo boas práticas de programação, com separação em camadas (UI, regras de negócio e persistência), facilitando manutenção e escalabilidade.

RNFS05	
Função	Banco relacional;
Descrição	Persistência em banco relacional com integridade (chaves, relacionamentos e validações);
Entradas	Dados do sistema (usuários, games, acessos, pacotes);
Fonte	Sistema, Usuário;
Saídas	Dados estruturados e consistentes, Integridade relacional;
Ação	O sistema deve utilizar banco de dados relacional, garantindo integridade, consistência e organização dos dados por meio de tabelas e relacionamentos.

RNFS06	
Função	Documentação;
Descrição	Documentação clara e padronizada do projeto e do código (README e manual rápido de execução);
Entradas	Informações do Sistema, códigos e regras de negócio;
Fonte	Desenvolvedor;
Saídas	Documentação técnica, Documentação de Uso;
Ação	O sistema deve possuir documentação clara, contendo descrição dos requisitos, estrutura do banco de dados, regras de negócio e instruções de uso.

4. CASOS DE USO

Apresentar 3 casos de uso do sistema

5. DIAGRAMA DE CLASSE

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema**:
- **Objetivo do Sistema**: (Breve descrição da finalidade do sistema)
- **Público-Alvo**:
- **Principais Funcionalidades**:

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades**:
  - Interface com o usuário
  - Validação inicial de dados
  - Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades**:
  - Implementação da lógica de negócio
  - Autenticação/autorização
  - Comunicação com a camada de dados
  - Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição**:
  - Contém as regras de negócio e os modelos de domínio
  - Independente das demais camadas
- **Componentes**:
  - Entidades
  - Casos de uso

- Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades**:
  - Armazenamento e recuperação de dados
  - Scripts de criação e migração de banco

6.4. Diagrama da Arquitetura

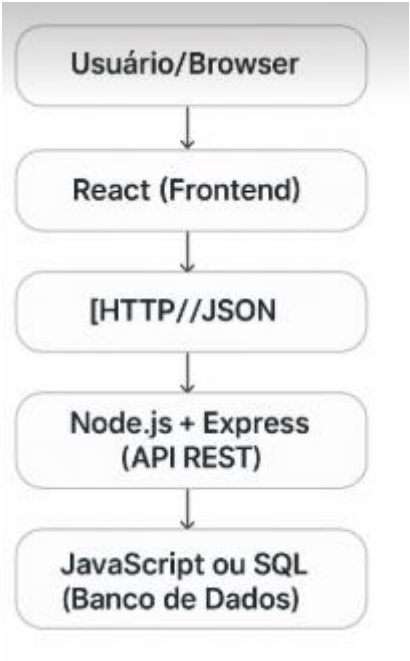
Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

```
[Usuário/Browser]
      |
    React (Frontend)
      |
   [HTTP/JSON]
      |
Node.js + Express (API REST)
      |
  JavaScript ou SQL
      |
 PostgreSQL (Banco de Dados)
```

Diagrama:



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```
🗀 /frontend
    🗀 components/
    🗀 pages/
    🗀 services/
🗀 /backend
    🗀 controllers/
    🗀 services/
    🗀 models/
    🗀 routes/
    🗀 config/
🗀 /database
    🗎 schema.sql
    🗎 migrations/
```

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes**: desenvolvimento, homologação, produção
- **Contêineres**: Docker/Docker Compose (se usado)

- **CI/CD**: GitHub Actions, GitLab CI, Jenkins
- **Hospedagem**: Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.